

JAVA

08

Spring WebServices & AI Integration

Call real-world AI APIs from Java — and build a chatbot that actually talks back.

DURATION

4 Sessions

LEVEL

Advanced

MODULE

Module 8

UPDATED

June 2026

IN THIS MODULE

01

Introduction to AI in Java

02

Working with REST APIs in Java

03

Integrating OpenAI API with Spring Boot

04

Building a Chatbot with LangChain4J

SESSION

01

90 MIN · AI INTEGRATION

Introduction to AI in Java

◆ Learning Objectives

- Explain what AI APIs are and how they differ from training your own machine learning model
- Compare OpenAI, Google Vertex AI, and Hugging Face as AI service providers for Java projects
- Use Java 11 HttpClient to send a synchronous HTTP POST request with authentication headers and a body
- Identify the five key components of any REST API call: method, endpoint, headers, body, and status code
- Locate the AI model reply inside an OpenAI Chat Completions JSON response at `choices[0].message.content`

01 The AI API Landscape

An AI API lets your Java application use a pre-trained machine learning model that someone else built and hosts — you send a JSON request over HTTP and receive an intelligent response. There is no model to train, no GPU to rent, and no data pipeline to build. Companies like Swiggy, PhonePe, and Zomato already use AI APIs in production to power smart restaurant recommendations, UPI fraud detection, and real-time regional language translation for Hindi, Gujarati, and Tamil speakers. Your first AI-powered Java feature is just one HTTP call away.

CONCEPT — AI API vs Training Your Own Model

Training a model means feeding terabytes of data into an algorithm and waiting days on expensive GPUs. Calling an AI API means sending a JSON request to a provider like OpenAI who has already done all that training — you pay per call with zero upfront investment. For 95% of Java projects, calling an API is the right choice. Custom training is only justified when you have proprietary private data or require total control that no provider offers.

INDIA AI INDUSTRY SIZE (2027 PROJECTION)

Rs 8.47L Cr

Source: NASSCOM AI Adoption in India Report, 2024

02 Major AI Service Providers

Provider	Flagship Model	Free Tier	Best For Java Devs
OpenAI	GPT-4o, GPT-3.5-turbo	Trial credits on sign-up	Chat, text generation, function calling

Provider	Flagship Model	Free Tier	Best For Java Devs
Google Vertex AI	Gemini 1.5 Pro / Flash	GCP free trial (\$300 credit)	Enterprise Spring Boot apps on Google Cloud
Hugging Face	Mistral-7B, Llama-3	Free Inference API (rate-limited)	Dev and testing — zero cost during build phase

For a new Java project in India, start with Hugging Face to validate your integration code for free, then move to OpenAI for production quality. Avoid locking into one provider too early — design your service layer so switching models is a config change, not a code rewrite. Every major Indian SaaS company from MoEngage to Freshworks follows this multi-model strategy in 2025. The Java HttpClient code you write today will work with any of these three providers; only the endpoint URL and auth header format change.

PRO TIP — Use Hugging Face for Free Development

Hugging Face's Inference API lets you call open-source models like Mistral-7B or Meta Llama-3 completely free during development. Build and test your full Java HTTP pipeline, handle all error scenarios, and validate your JSON parsing — then switch to a paid OpenAI key only when you are ready for production quality. The HttpClient code is identical; only the URL, auth header value, and response JSON structure change slightly.

03 Java's HTTP Stack

Java 11 introduced a built-in HTTP client in the `java.net.http` package that replaces the old, verbose `URLConnection` API. It offers a clean fluent builder, native async support via `CompletableFuture`, and easy timeout and redirect configuration — all without any external dependency. Every modern Spring Boot project on Java 17 or higher should use this client. You used it in earlier modules to call REST APIs; now you will use the exact same code to call AI model APIs.

URLConnection (Legacy)

Available since Java 1.1. Requires manual stream opening and closing. No built-in async or timeout API. Verbose — up to 20 lines for a basic POST. Still works, but not recommended for any new code written after 2019.

Java 11 HttpClient (Modern)

Built into Java 11+. Fluent builder — concise and readable. Native async with `CompletableFuture`. Clean timeout and SSL configuration. No extra JAR dependency. The correct choice for all new Java projects calling external REST or AI APIs.

```
// Declare HttpClient as a singleton – one instance per app
// Create it once; reuse across all requests
HttpClient client = HttpClient.newBuilder()    Fluent builder — chain config calls before build()
    .connectTimeout(Duration.ofSeconds(10)).build();
// Fail fast if the AI server does not respond in 10s
// Build the HTTP request
```

```
HttpRequest request = HttpRequest.newBuilder()
    Immutable request — built once, sent once
    .uri(URI.create(endpoint))                endpoint = the AI provider URL loaded from config
    .header("Authorization", "Bearer " + apiKey)
    All major AI APIs use Bearer token authentication
    .header("Content-Type", "application/json")
    Required when sending JSON in the request body
    .POST(HttpRequest.BodyPublishers.ofString(jsonBody)).build();
    jsonBody = serialized request (covered in Session 2)
// Send synchronously and inspect the response
HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
    Blocks until the response arrives
System.out.println("Status: " + response.statusCode());
    200 = OK, 401 = bad key, 429 = rate limited, 500 = down
System.out.println(response.body());          Prints the full raw JSON returned by the AI provider
```

04 Anatomy of an OpenAI API Request

Every OpenAI Chat Completions call sends a JSON object with two required fields: model and messages. The messages field is an array where each item has a role (system, user, or assistant) and a content string. The system role sets the AI's permanent persona — it always follows this instruction regardless of what the user says. The user role is what the end-user typed. OpenAI returns a JSON object with a choices array; the AI's actual reply lives at choices[0].message.content. Every Java class you build in Sessions 2 and 3 will map to this exact structure.

JSON Field	Type	Purpose
model	String	Which AI model to use — e.g. gpt-3.5-turbo or gpt-4o
messages	Array	Conversation history — each item has a role and content
messages[].role	String	system = persona; user = student input; assistant = past AI reply
messages[].content	String	The actual text for that message
max_tokens	Integer (optional)	Maximum reply length — set this to control cost per API call
temperature	Float 0–2 (optional)	0 = deterministic output; 1 = balanced; 2 = very creative

TRY THIS LIVE

Open platform.openai.com/playground. Set the system message to: 'You are a helpful course advisor at a Java training institute in Ahmedabad.' Type 'What job can I get after Java Full Stack?' in the user box and run it. Switch to the JSON view — find `choices[0].message.content`. That path is exactly what your Java code will navigate when parsing the response.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — IIT Kharagpur; covers Java I/O, networking fundamentals, and the HTTP communication stack that underpins `HttpClient`

Link — <https://nptel.ac.in/courses/106105191>

Always link the official channel — beware fan-made or re-upload copies.

WHERE TO GO NEXT

You can now make a valid HTTP call to any AI API and understand its response structure — the foundation every remaining session in this module builds on.

Explore deeper — OpenAI API Reference at platform.openai.com/docs/api-reference/chat — read the Chat Completions section end to end before Session 2

Try on your own — Call the Hugging Face Inference API with model `mistralai/Mistral-7B-Instruct-v0.2` using the same `HttpClient` setup and print the raw JSON response to the console

Connect the dots — Session 2 replaces raw JSON string building with Jackson `ObjectMapper`, adds proper HTTP error handling, and maps the AI response to typed Java POJOs

Going further — Read about OpenAI function calling — it lets the AI model call your Java methods by name, enabling AI-driven workflows far beyond simple text generation

SESSION

02

90 MIN · REST API DEVELOPMENT

Working with REST APIs in Java

◆ Learning Objectives

- Send GET and POST HTTP requests using Java 11 HttpClient with correct authentication headers
- Use Jackson ObjectMapper to serialize Java objects into properly escaped JSON strings
- Deserialize AI API JSON responses into typed Java POJOs using ObjectMapper.readValue
- Identify HTTP error status codes returned by AI and payment APIs and write appropriate handling logic
- Store API keys securely using environment variables rather than hardcoded string literals in source code

01 REST APIs: The Language of the Modern Web

Every significant backend feature a Java developer builds in 2025 requires consuming at least one external REST API — Razorpay for payments, MSG91 for OTP, SendGrid for email, Aadhaar for identity, or OpenAI for intelligence. REST is the universal contract: the client sends an HTTP request with a method, endpoint, headers, and optional body; the server returns a status code and a JSON response. Mastering REST consumption in Java is as fundamental as mastering OOP — it is the skill that connects your Spring Boot backend to everything else in the modern software ecosystem.

CONCEPT — The Five Parts of Every REST API Call

1. Method — GET to read, POST to create or process, PUT to update, DELETE to remove. 2. Endpoint URL — the address of the resource. 3. Headers — metadata like Authorization (carries the API key) and Content-Type (tells the server you are sending JSON). 4. Body — the JSON data you send with POST or PUT. 5. Status Code — the server's verdict: 200 is success, 401 is wrong credentials, 429 is rate limit exceeded, 500 is server error. Your Java code must always check the status code before touching the response body.

02 Java 11 HttpClient: Complete Walkthrough

HttpClient is designed to be created once and reused across all requests — it internally manages a connection pool, so creating a new instance per request wastes memory and is significantly slower. In Spring Boot, declare HttpClient as a @Bean in a @Configuration class and inject it with @Autowired into every service that makes HTTP calls. Set connectTimeout at the builder level so every request through this client has an automatic timeout — this one line prevents production incidents where a bank or AI provider stops responding and your threads hang indefinitely.

```
// --- GET Request: read a resource ---
HttpRequest getReq = HttpRequest.newBuilder()
    .uri(URI.create("https://api.publicapis.org/entries"))
    .build();
```

No request body; data is in URL or query params
Free public API — no key needed, good for testing

```
.GET().timeout(Duration.ofSeconds(5)).build();
    GET method; fail if server does not respond in 5s
HttpResponse<String> getRes = client.send(getReq, BodyHandlers.ofString());
    Execute synchronously; response body is a String
// --- POST Request: send data to create or process ---
    All AI API calls are POST — you send the prompt
HttpRequest postReq = HttpRequest.newBuilder()
    .uri(URI.create("https://api.openai.com/v1/chat/completions"))
    .header("Authorization", "Bearer " + System.getenv("OPENAI_API_KEY"))
    .header("Content-Type", "application/json")
    .POST(BodyPublishers.ofString(jsonBody)).build();
    jsonBody = Jackson-serialized ChatRequest DTO
HttpResponse<String> postRes = client.send(postReq, BodyHandlers.ofString());
    Always check postRes.statusCode() before parsing body
```

PRO TIP — One HttpClient Bean for the Whole App

In your Spring Boot @Configuration class, declare HttpClient as a @Bean using `HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(10)).build()`. Inject this bean with @Autowired in every service class that makes external calls — OpenAIService, PaymentService, SMSService. One shared connection pool, timeout set in exactly one place, and every service benefits automatically. This is the pattern used in every production-grade Java backend in India.

03 JSON Serialization with Jackson ObjectMapper

Building JSON by hand using string concatenation breaks the moment a user types a double quote, a newline character, or a Hindi letter like " in their input — the manually concatenated string becomes invalid JSON and your API call fails with a 400 error. Jackson's ObjectMapper solves this completely: `writeValueAsString` converts any Java object to a safe, properly escaped JSON string; `readValue` maps a JSON string back to a typed Java class. Jackson is bundled inside every Spring Boot project — no extra dependency needed.

◆ Java -> JSON -> Java: The Full Round Trip

```
// Step 1 — Define DTO classes (one per JSON structure)
    Jackson maps Java field names to JSON keys by default
public class ChatRequest {
    private String model;
    private List<ChatMessage> messages;
```

Represents the JSON body you send to OpenAI
Maps to JSON key 'model'
Maps to JSON array 'messages'

```
// Add constructor, getters, and setters
}
// Step 2 – Serialize: Java object to JSON string
    Use before sending any API request
    ObjectMapper mapper = new ObjectMapper();
    String jsonBody = mapper.writeValueAsString(chatRequest);
    // Step 3 – Deserialize: JSON string to Java object
    Use after receiving the API response
    ChatResponse chatResponse = mapper.readValue(responseBody, ChatResponse.class);
    String reply = chatResponse.getChoices().get(0).getMessage().getContent();
```

Jackson needs getters to serialize, setters to deserialize

Thread-safe — declare as a @Bean and reuse everywhere

Handles all escaping including Hindi and special chars

Maps every JSON key to the matching Java field name

Navigate to choices[0].message.content for the AI reply

TRY THIS LIVE

Create a ChatRequest class with model (String) and messages (List) fields. Create a ChatMessage class with role and content fields. Instantiate a ChatRequest for gpt-3.5-turbo with one user message asking 'What is polymorphism in Java?' Run mapper.writeValueAsString() on it and paste the output into jsonlint.com to confirm it is valid JSON. This is the exact body your POST request will send to OpenAI.

04 API Error Handling and Key Security

Most Indian developers get burned by two things in their first production AI integration: unhandled HTTP error codes and accidentally exposed API keys. AI APIs return very different status codes for different failure modes — a 429 Too Many Requests needs a timed retry with backoff, but a 401 Unauthorized should immediately trigger an alert to the team. Always check the status code before calling ObjectMapper — a 429 response body is an error JSON, not a ChatResponse, and readValue will throw an exception if you try to map it to the wrong class.

HTTP Status	Meaning	Java Handling Strategy
200 OK	Request succeeded	Parse response body with ObjectMapper normally
400 Bad Request	Malformed JSON or invalid parameters sent	Log the request body; fix the DTO structure
401 Unauthorized	API key missing, expired, or invalid	Throw AuthenticationException; do not retry
429 Too Many Requests	Rate limit exceeded	Wait and retry with exponential backoff: 1s, 2s, 4s

HTTP Status	Meaning	Java Handling Strategy
500 Internal Server Error	AI provider servers are down	Catch, log, return a graceful fallback message to the user

WARNING — Never Hardcode API Keys

Writing your API key as a plain string in Java source code means it gets committed to Git and is permanently visible to anyone with repository access — including public GitHub. Use `System.getenv()` to read the key from an OS environment variable, or use Spring Boot's `@Value` annotation with the actual key stored in an `.env` file that is listed in `.gitignore`. Exposed API keys on public GitHub are scanned and exploited by automated bots within minutes — this is not theoretical, it happens daily.

CASE STUDY — Razorpay API Error Normalisation

Context — Razorpay is India's largest payment gateway, processing transactions for over 8 million businesses — from small Ahmedabad kirana stores to IRCTC and Zomato — totalling over Rs 5 lakh crore per year.

Challenge — When Razorpay calls upstream bank APIs (HDFC, SBI, ICICI) and UPI switches to process a payment, each bank returns its own inconsistent error format — different HTTP status codes, different JSON structures, and different messages for what is essentially the same failure type.

What they did — Razorpay built an internal error normalisation layer in their Java backend that reads each upstream API's raw HTTP response, maps it to a consistent internal error code catalog, and returns one predictable JSON error structure to every developer integrating Razorpay, regardless of which bank failed underneath.

Result — Any developer consuming the Razorpay API writes a single error-handling block that works across all payment methods and all bank providers. Developer onboarding time for new payment integrations dropped significantly, and support ticket volume from confused integrators fell.

Takeaway — The HTTP error code handling table you just studied is not academic — it is exactly the architecture Razorpay's Java engineers built to make their API reliable for 8 million Indian merchants.

Source: Razorpay Engineering Blog — engineering.razorpay.com; Razorpay API Documentation, 2024

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a senior Java backend developer at a Bangalore startup. Explain in simple Hinglish how to securely store and use an OpenAI API key in a Spring Boot project — covering `System.getenv()`, the `@Value` annotation, and what actually happens when a developer accidentally pushes the key to a public GitHub repo. Use a small Ahmedabad SaaS startup as the example scenario. Keep it under 150 words.

SESSION

03

90 MIN · SPRING BOOT + AI

Integrating OpenAI API with Spring Boot

◆ Learning Objectives

- Set up a Spring Boot project with the correct dependencies to call the OpenAI API
- Bind all OpenAI configuration values cleanly using `@ConfigurationProperties`
- Build an `OpenAIService` class that encapsulates `HttpClient` and `Jackson` in a reusable service layer
- Design `ChatRequest`, `ChatMessage`, and `ChatResponse` DTO classes that map to the OpenAI JSON schema
- Expose a `POST /api/ask` endpoint that accepts a user question and returns an AI-generated reply

01 Spring Boot as an AI Gateway

In a full-stack application, the Spring Boot backend sits between the React frontend and the OpenAI API — acting as a secure gateway. The frontend never holds the API key; the Spring Boot service does. When a student asks a question on the TOPS campus portal, React sends a `POST` to your Spring Boot `/api/ask` endpoint. Spring Boot adds the secret API key, calls OpenAI, receives the reply, and returns it to React. This three-layer architecture keeps credentials safe and gives you full control over rate limiting, input validation, prompt engineering, and fallback logic when OpenAI is down.

CONCEPT — Spring Boot as a Secure AI Proxy

If your React frontend called OpenAI directly, the API key would be visible in the browser network tab — any user could extract it and run up your bill within minutes. The Spring Boot backend acts as a proxy: the frontend sends a clean request to your own `/api/ask` endpoint, Spring Boot adds the secret API key server-side, calls OpenAI, and returns only the AI reply to the client. All authentication, rate limiting, and prompt engineering logic lives safely on the server.

02 Project Setup and Configuration

Create a new Spring Boot project at start.spring.io selecting Spring Web and Validation as dependencies — no special AI library is needed for this session because you call OpenAI using the built-in Java 11 `HttpClient`. Create a dedicated `OpenAIConfig` class annotated with `@ConfigurationProperties` to bind all `openai.*` properties from `application.properties` into one typed object. This is cleaner than scattering `@Value` annotations across multiple service classes: all settings are visible in one place, easily testable, and changing a property name means editing one class, not hunting through the whole project.

application.properties — all OpenAI settings in one place

Reference env variable so the real key never enters the codebase

openai.api.key=\${OPENAI_API_KEY}

Reads from OS environment variable at application startup

openai.model=gpt-3.5-turbo

Change to gpt-4o here — no code change needed anywhere else

openai.base-url=https://api.openai.com/v1/chat/completions

Full endpoint URL — swap this to change AI providers

openai.max-tokens=500

Cap reply length — directly controls cost per API call

----- OpenAIConfig.java -----

Companion class that binds all openai.* properties

@ConfigurationProperties(prefix = "openai")

Maps every openai.* key to a matching Java field

@Component

Spring manages this bean — inject anywhere with

@Autowired

```
public class OpenAIConfig {
```

```
    private String apiKey, model, baseUrl;
```

Field names match property keys (camelCase = kebab-case)

```
    private int maxTokens;
```

```
// Add standard getters and setters for all fields
```

Spring needs setters to inject; services need getters to read

```
}
```

PRO TIP — @ConfigurationProperties over @Value

Using @Value to inject individual properties in multiple service classes fragments your configuration — it is scattered, hard to test, and any typo silently injects null at runtime. Grouping all related settings in a single @ConfigurationProperties class gives you one authoritative location, full IDE autocomplete for property names, and a class you can easily mock in unit tests. This is the standard pattern at every professional Spring Boot shop in India.

03 Building the OpenAI Service Layer

The service layer is where the actual API call logic lives. OpenAIService is annotated with @Service and injected with OpenAIConfig (all settings), HttpClient (HTTP transport), and ObjectMapper (JSON). The single public method ask(String question) does five things in sequence: builds a ChatRequest DTO, serializes it to JSON, sends an HTTP POST to OpenAI, checks the status code, and returns the reply text. Controllers call this method and get a String back — they never see an HTTP request, a JSON document, or an API key.

@Service

Spring-managed service bean — inject into any controller

```
public class OpenAIService {
```

```
    @Autowired private OpenAIConfig config;
```

All OpenAI settings loaded from application.properties

```

    @Autowired private HttpClient httpClient;    Shared singleton client — one connection pool for the
                                                app
    @Autowired private ObjectMapper mapper;      Jackson — already a Spring bean in every Boot project
    public String ask(String question) throws Exception {
        One public method — question in, AI reply text out
        ChatRequest req = buildRequest(config.getModel(), question);
        Assembles DTO with system message + user question
        String jsonBody = mapper.writeValueAsString(req);
        Serializes DTO safely — handles all special chars and Hindi
        HttpRequest httpReq = HttpRequest.newBuilder()
            .uri(URI.create(config.getBaseUrl()))
            Endpoint from config — never hardcoded in service
            .header("Authorization", "Bearer " + config.getApiKey())
            API key injected from secure config object
            .header("Content-Type", "application/json")
            Required by OpenAI for all POST requests
            .POST(BodyPublishers.ofString(jsonBody)).build();
        HttpResponse<String> res = httpClient.send(httpReq, BodyHandlers.ofString());
        if (res.statusCode() != 200)              Always verify status before parsing body
        throw new RuntimeException("OpenAI error: " + res.statusCode());
        Let the controller handle the error response
        return mapper.readValue(res.body(), ChatResponse.class)
        Deserialize full response
        .getChoices().get(0).getMessage().getContent();
        Navigate choices[0].message.content for the AI reply
    }
}

```

```

// ChatRequest.java — the JSON body sent to OpenAI
public class ChatRequest {
    private String model;                e.g. gpt-3.5-turbo or gpt-4o
    private List<ChatMessage> messages;  Conversation history: system + user messages
    private int maxTokens;              Jackson maps this to max_tokens — use
                                        @JsonProperty if needed
}
// ChatMessage.java — one item in the messages array
public class ChatMessage {
    private String role;                 system | user | assistant
    private String content;              The text for this message
}
// ChatResponse.java — the JSON object OpenAI returns
public class ChatResponse {
    private List<Choice> choices;        OpenAI always returns a list — usually one item
}

```

```
// Choice.java – one item in choices[]
public class Choice { private Message message; }
    choices[0].message
// Message.java – the AI reply
public class Message { private String content; }
    The actual AI reply text lives here
```

TRY THIS LIVE

Create a CommandLineRunner bean in your Spring Boot app, inject OpenAIService, and call openAIService.ask("What are the top 3 Java job roles in Ahmedabad right now?"). Run the project and read the AI reply in the console. This verifies your config binding, service layer, and HTTP call are all working before you wire up any REST controller.

04 The /api/ask REST Endpoint

The AskController is intentionally thin — it validates the incoming request, delegates all business logic to OpenAIService, and formats the response. Controllers must never contain HTTP client logic, JSON parsing, or API key handling. Keeping the controller thin makes it easy to unit-test with Spring's MockMvc and easy to swap the AI provider without touching the controller. The endpoint accepts JSON with a question field and returns JSON with a reply field — a simple, predictable contract the React frontend can rely on.

```
@RestController
// Combines @Controller + @ResponseBody — returns JSON by default

@RequestMapping("/api")
// All endpoints in this class are under the /api prefix
public class AskController {
    @Autowired private OpenAIService openAIService;
    // Inject the service — never instantiate with new

    @PostMapping("/ask")
    // Handles POST /api/ask — test with Postman
    public ResponseEntity<Map<String, String>> ask(
        @RequestBody Map<String, String> body) {
        // Accepts JSON body: {"question": "..."}
        String question = body.get("question");
        // Extract the user's question from the request JSON
        if (question == null || question.isBlank())
            // Always validate before calling a paid AI API
            return ResponseEntity.badRequest()
                .body(Map.of("error", "Question cannot be empty"));
        // Return HTTP 400 with a descriptive error message
        try {
            String reply = openAIService.ask(question);
            // Delegate to service — controller stays thin
```

```
return ResponseEntity.ok(Map.of("reply", reply));
    Return 200 with {"reply": "...AI answer..."}
    } catch (Exception e) {
return ResponseEntity.internalServerError()
    .body(Map.of("error", "AI service temporarily unavailable"));
    Never expose the raw exception message to the client
    }
    }
}
```

CASE STUDY — Freshworks Freddy AI

Context — Freshworks is a Chennai-headquartered SaaS company listed on NASDAQ (FRSH), building CRM and customer support tools used by 68,000+ businesses globally — including Zomato, Dunzo, and Urban Company.

Challenge — Customer support agents at Freshworks customers were spending over 70% of their time resolving repetitive tickets — password resets, billing queries, and order status checks. The same answers were being typed manually thousands of times every day.

What they did — Freshworks built Freddy AI on a Spring Boot backend that calls OpenAI GPT APIs through a secure service layer. Freddy auto-classifies incoming support tickets, suggests reply templates to agents, and fully resolves simple queries without any human involvement — the exact architecture built in this session.

Result — Customers using Freddy AI resolve over 40% of support tickets automatically, with no agent action required. Human agents now focus exclusively on complex conversations that require empathy and judgment.

Takeaway — The OpenAIService and AskController you built today are architecturally identical to what Freshworks built for a product serving 68,000 businesses — the scale differs, not the Spring Boot pattern.

Source: Freshworks Freddy AI product documentation at freshworks.com/freddy-ai; Freshworks Investor Relations Report, 2024

“The best AI feature is invisible — users just experience a product that is smarter than they expected.”

WHERE TO GO NEXT

You now have a working Spring Boot endpoint that calls OpenAI and returns an AI reply — a deployable, production-ready integration that any frontend can consume.

Explore deeper — Spring Boot documentation on `@ConfigurationProperties` at docs.spring.io — read the type-safe configuration properties section for advanced binding options

Try on your own — Add a second endpoint POST /api/summarise that takes a long text body and instructs OpenAI to summarise it in three bullet points — reuse the same OpenAIService with no changes

Connect the dots — Session 4 replaces the raw HttpClient approach with LangChain4J — adding prompt templates, conversation memory, and a cleaner annotation-based interface for building a full chatbot

Going further — Read about Spring AI at spring.io/projects/spring-ai — the official Spring abstraction for AI models that standardises the pattern you built manually in this session

SESSION

04

90 MIN · CHATBOT DEVELOPMENT

Building a Chatbot with LangChain4J

◆ Learning Objectives

- Explain what LangChain4J is and why it simplifies AI application development compared to raw API calls
- Add the LangChain4J OpenAI Spring Boot Starter to a Maven project and configure it via application.properties
- Define a type-safe AI service interface using the @SystemMessage and @UserMessage annotations
- Create reusable prompt templates with dynamic variable injection using the {{variable}} syntax and @V
- Attach conversation memory using MessageWindowChatMemory so the chatbot retains context across turns

01 From Raw API Calls to AI Applications

The HttpClient-based integration built in Session 3 works for a single use case. But when your app needs five distinct AI features — a course advisor, a code reviewer, an essay grader, an FAQ bot, and a language translator — each with its own prompt persona, each needing conversation history, and each potentially running on a different model, raw API calls become unmanageable boilerplate. LangChain4J is a Java-native framework that gives you all of this in clean, annotation-based interfaces. It is to LLM development what Spring Data JPA is to database development: an abstraction that handles all the plumbing so you focus entirely on the logic.

Raw HttpClient + Jackson

You manage HTTP requests, auth headers, JSON serialization, response parsing, error codes, status checks, and prompt string construction manually. Works for one or two use cases. Becomes fragile and repetitive when you have five or more AI features, multiple models, or conversation history to manage.

LangChain4J AiServices

Define a Java interface. Add annotations. Call your method. LangChain4J handles HTTP, JSON, prompt construction, model switching, and conversation memory automatically — behind a clean Java API that is easy to read, test, mock, and refactor.

CONCEPT — LangChain4J: Java-Native LLM Framework

LangChain4J (langchain4j.dev) is the Java equivalent of Python's LangChain library. Its core feature is AiServices: you declare a Java interface with annotated methods, call AiServices.create(), and receive back a fully working implementation. You never write an HttpRequest, serialize a ChatRequest, or parse a JSON response manually — you just call a method on an interface, the same way you call a Spring Data JPA repository method to query a database.

02 Setting Up LangChain4J in Spring Boot

LangChain4J provides dedicated Spring Boot starters for each AI provider so setup is minimal. Add langchain4j-open-ai-spring-boot-starter to pom.xml and set the API key property in application.properties using the same environment variable pattern from Session 3. The starter auto-configures an OpenAiChatModel bean that

Spring injects automatically. LangChain4J also supports Google Gemini, Azure OpenAI, Hugging Face, Anthropic Claude, and local Ollama models — switching providers later is a dependency swap and a properties change, not a service rewrite.

```
<!-- pom.xml – add the LangChain4J OpenAI Spring Boot Starter -->
    Check mvnrepository.com for the latest stable version
<dependency>
    <groupId>dev.langchain4j</groupId>
<artifactId>langchain4j-open-ai-spring-boot-starter</artifactId>
    Includes auto-configuration for OpenAiChatModel bean
    <version>0.36.0</version>                                Verify the latest version before use
</dependency>
# application.properties – LangChain4J reads these automatically
langchain4j.open-ai.chat-model.api-key=${OPENAI_API_KEY}
    Same env variable pattern as Session 3
langchain4j.open-ai.chat-model.model-name=gpt-3.5-turbo
// Define your AI feature as a plain Java interface
    LangChain4J auto-generates the implementation at runtime
public interface CampusBotService {
    @SystemMessage("You are a course advisor at TOPS Technologies, Ahmedabad.")
        Permanent AI persona applied automatically to every call
    String answer(@UserMessage String question);
        @UserMessage injects the parameter as the user message
}
// Register as a Spring bean in your @Configuration class
@Bean CampusBotService campusBot(OpenAiChatModel model) {
    Spring injects the auto-configured OpenAiChatModel
    return AiServices.create(CampusBotService.class, model);
        LangChain4J creates a proxy — no HTTP code written by you
}
```

PRO TIP — All AI Logic as @AiService Interfaces

Every AI-powered feature in your app — course advisor, code reviewer, essay grader, FAQ bot — should be a separate @AiService interface with its own @SystemMessage persona. This keeps prompt logic out of controllers and services, makes prompts visible and editable in one place, and lets you unit test by injecting a mock implementation. This is the same separation-of-concerns principle as the Repository pattern, applied to AI interactions.

03 Prompt Templates and System Messages

A hardcoded prompt string built with string concatenation is fragile — it fails on special characters, is invisible to your team, cannot be reused, and mixes business logic with infrastructure code. LangChain4J prompt templates use {{variable}} double-brace syntax for dynamic injection, with @V mapping Java method parameters to template slots.

The `@SystemMessage` defines the AI's permanent persona and scope restrictions for the entire interface. A well-crafted system message for a TOPS course advisor ensures the AI stays on topic, recommends demo classes, and never gives advice outside the domain you define.

```
public interface CourseAdvisorService {
    @SystemMessage("You are an expert course advisor at TOPS Technologies, Ahmedabad.")
    Permanent persona — all calls use this system message
    @UserMessage("Student from {{city}} asks: {{question}}")
    Template with two dynamic variables — injected at call time
    String advise(@V("city") String city, @V("question") String question);
    @V maps Java params to {{variable}} slots in the template
}
// Register as a Spring @Bean (same pattern as CampusBotService)
@Bean CourseAdvisorService courseAdvisor(OpenAiChatModel model) {
    return AiServices.create(CourseAdvisorService.class, model);
}
// Call from a @RestController — LangChain4J builds the prompt automatically
String reply = courseAdvisor.advise("Surat", "Which course gets me a job fastest?");
LangChain4J injects Surat and the question into the template
```

TRY THIS LIVE

Create a `TOPSAdvisorService` interface with a `@SystemMessage` that says: 'You are a friendly course advisor at TOPS Technologies, Ahmedabad. Only answer questions about Java Full Stack, Python, React, and Android courses. Always mention that a free demo class is available.' Add a method with a `@UserMessage` that takes one question parameter. Register it as a Spring bean and call it from a `CommandLineRunner` with the question: 'I know basic HTML — which course should I take next?' Print the reply.

04 Adding Conversation Memory

A stateless chatbot is frustrating: the user says 'My name is Priya, I am from Surat' and two turns later asks 'What courses suit me?' — a stateless bot has no idea who Priya is or where she is from and gives a generic answer. LangChain4J's `ChatMemory` stores the full conversation history and includes it automatically in every subsequent prompt. `MessageWindowChatMemory` keeps the last N messages, ensuring the context window never overflows even in long conversations. In a Spring Boot app, scope each `ChatMemory` to the HTTP session — every logged-in user gets their own isolated conversation history.

```
// Create a memory window — retains the last N messages
Do NOT share one ChatMemory instance across multiple users
ChatMemory memory = MessageWindowChatMemory.withMaxMessages(10);
Keeps last 10 messages — 5 user turns + 5 AI replies
```

```
// Use AiServices.builder() when you need memory or extra tools
CampusBotService bot = AiServices.builder(CampusBotService.class)
    .chatLanguageModel(openAiChatModel)           The Spring-injected OpenAiChatModel bean
    .chatMemory(memory)                          Attach memory — bot now remembers the
                                                conversation

    .build();
// Demonstrate memory across two turns
String r1 = bot.answer("My name is Arjun and I live in Vadodara.");
    Bot stores this context in memory
String r2 = bot.answer("What Java course do you recommend for me?");
    Bot recalls Arjun and Vadodara and personalises the reply
// Spring Boot: one memory instance per HTTP session
@SessionScope @Bean CampusBotService sessionBot(OpenAiChatModel m) {
    @SessionScope creates a new bean instance per HTTP session
    ChatMemory mem = MessageWindowChatMemory.withMaxMessages(20);
    return AiServices.builder(CampusBotService.class)
        .chatLanguageModel(m).chatMemory(mem).build();
    Each logged-in student gets their own private memory
}
```

CASE STUDY — Haptik (JIO): India's Largest Chatbot Platform

Context — Haptik is a Mumbai-based conversational AI company acquired by Reliance JIO in 2019 for approximately \$100 million. Their platform now serves over 500 million JIO subscribers across India.

Challenge — JIO MyJio app needed a chatbot capable of handling millions of queries per day in both English and Hindi — about recharges, data plans, network issues, and billing — without a human agent. Each conversation needed memory to feel natural across multiple user turns.

What they did — Haptik built a prompt-template-driven chatbot system where each domain (billing, network, plans, troubleshooting) has its own system message defining scope and persona. Conversation memory stores the last several user-bot exchanges so the assistant gives contextual replies — architecturally identical to the LangChain4J pattern taught in this session.

Result — JIO's virtual assistant handles millions of conversations per day, reducing call centre load by over 30% and resolving routine queries in under 10 seconds — at JIO's scale of more than 500 million subscribers.

Takeaway — The @SystemMessage per use case, prompt templates with dynamic variables, and per-session ChatMemory you built in this session are exactly the architectural patterns powering India's largest chatbot deployment.

Source: Economic Times — Reliance JIO acquires Haptik for ~\$100M, 2019; Haptik.ai product documentation, 2024

“A chatbot without memory is just a fancy search bar — with memory, it becomes a real conversation.”

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a LangChain4J expert who teaches Java developers in India. Explain in simple Hinglish the difference between calling OpenAI directly with HttpClient versus using LangChain4J AiServices — focus specifically on what LangChain4J saves the developer from doing manually. Use the TOPS Technologies campus chatbot (CampusBot) as the running example. Keep it under 150 words.

REVIEW



INTERVIEW PREP · MODULE 8

Interview Questions

◆ Spring WebServices

Rehearse each answer out loud and aim for a 60-second response. Use the hint only as a nudge if you are stuck. Compare your attempt with the model answer afterward to find gaps.

1

What is an API, and why would a Java application use an AI API such as OpenAI or Google Vertex AI?

Think about how external services provide functionality to your app.

MODEL ANSWER

An API (Application Programming Interface) is a contract that lets two software systems communicate — one system exposes endpoints, another calls them to request data or trigger actions. Java applications use AI APIs like OpenAI because training and running large language models requires enormous infrastructure that is impractical to own. Instead, your application sends a prompt over HTTP to OpenAI's endpoint and receives a generated response. This lets you add intelligent features — chatbots, text summarisation, code generation — with just a few lines of HTTP code.

2

Name two popular AI APIs that can be accessed from Java and mention one use case for each.

MODEL ANSWER

OpenAI API: used to generate natural-language answers to user questions — for example, a campus FAQ chatbot that answers queries about admission, fees, and events. Google Vertex AI (Gemini): used for multimodal tasks such as analysing uploaded images or documents alongside text — for example, a resume parser that reads a PDF and extracts structured candidate information. Both are called over HTTPS using standard Java HTTP libraries, returning JSON responses that your application parses and uses.

3

What is the purpose of making a REST call from Java to an AI service like OpenAI's API?

Consider what happens when your app needs to generate or analyze text using AI.

MODEL ANSWER

Your Java application cannot generate language natively — it needs to delegate that task to a model running on OpenAI's infrastructure. A REST call sends a JSON payload (model name, messages, parameters) over HTTPS to OpenAI's completions endpoint and receives a JSON response containing the model's generated text. This lets your Spring Boot backend act as a bridge: it accepts a user question, enriches or validates it, forwards it to OpenAI, and returns the answer — keeping API keys and business logic on the server, not exposed to the browser.

4

Explain the role of the Java HttpClient when working with REST APIs.

MODEL ANSWER

Java's HttpClient (introduced in Java 11) is a built-in library for sending HTTP requests and receiving responses. You build an HttpRequest with the URL, method (GET/POST), headers (Content-Type, Authorization), and body, then call 'client.send(request, BodyHandlers.ofString())' to get an HttpResponse. The response object contains the status code and the response body as a String. For OpenAI calls, you POST a JSON body and parse the returned JSON. HttpClient supports both synchronous and asynchronous (CompletableFuture-based) calls.

5

How would you use HttpClient in Java to make a GET request to an external API? Briefly describe the steps.

Focus on creating the client, building the request, sending it, and handling the response.

MODEL ANSWER

Create the client: 'HttpClient client = HttpClient.newHttpClient()'. Build the request: 'HttpRequest request = HttpRequest.newBuilder().uri(URI.create("https://api.example.com/data")).header("Accept", "application/json").GET().build()'. Send it synchronously: 'HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString())'. Check the status: 'if (response.statusCode() == 200)'. Parse the body: 'response.body()' returns the JSON string, which you then deserialise with Jackson or Gson.

6

What is the Jackson ObjectMapper used for in Java REST API integrations?

Think about converting between JSON and Java objects.

MODEL ANSWER

Jackson's ObjectMapper converts (serialises) Java objects to JSON strings and converts (deserialises) JSON strings back to Java objects. When calling an AI API, you use ObjectMapper to build the request body: 'objectMapper.writeValueAsString(requestMap)'. When you receive the response JSON, you use 'objectMapper.readTree(jsonString)' to navigate it as a tree, or 'objectMapper.readValue(jsonString, MyClass.class)' to deserialise it directly into a typed Java object. Jackson is the default JSON library in Spring Boot and handles nested objects, lists, and custom date formats automatically.

7

Suppose you receive a JSON response from an API containing a field called 'quote'. Write a simple Java code snippet using Jackson ObjectMapper to extract the value of 'quote' from the JSON string.

You might use ObjectMapper's readTree method and access the field.

MODEL ANSWER

'ObjectMapper mapper = new ObjectMapper(); JsonNode root = mapper.readTree(jsonResponse); String quote = root.get("quote").asText(); System.out.println(quote);'. readTree parses the JSON into a tree of JsonNode objects without needing a corresponding Java class. get("quote") navigates to the field, and asText() returns it as a String. If the field is nested (e.g., 'data.quote'), chain gets: 'root.get("data").get("quote").asText()'. Wrap in try-catch for JsonProcessingException.

8

How would you handle errors or unexpected responses when making API calls in Java?

Consider exceptions, status codes, or response validation.

MODEL ANSWER

Wrap the HttpClient call in a try-catch block to handle IOException and InterruptedException (network or thread errors). After receiving the response, check the status code: if it is not 200/201, read the error body and throw a custom exception or return a fallback message. For OpenAI, a 429 (rate limit) or 503 (overload) may need a retry with exponential backoff. Log the full request and response for debugging. Never surface raw API error messages directly to the user — translate them to user-friendly messages.

9

Describe how you would fetch an AI-powered 'quote of the day' from a public API and display it in your Java application.

MODEL ANSWER

Create an HttpClient and send a GET to 'https://api.quotable.io/random'. Parse the JSON response with Jackson to extract the 'content' and 'author' fields. In a Spring Boot application, create a QuoteService that performs this call and a QuoteController with '@GetMapping("/quote")' that returns the result. Cache the result for the day using @Cacheable (Spring Cache) so you do not call the external API on every request. Display it in the frontend by fetching '/quote' via Axios in React.

10

What are the main steps to integrate the OpenAI API with a Spring Boot application?

Think about configuration, authentication, and making the API call.

MODEL ANSWER

Add OpenAI API key to application.properties: 'openai.api.key=\${OPENAI_API_KEY}'. Create an OpenAIService class injecting the key with @Value. In the service, build the JSON request body (model, messages array) and send it via HttpClient to 'https://api.openai.com/v1/chat/completions' with header 'Authorization: Bearer {key}'. Parse the response JSON to extract 'choices[0].message.content'. Expose it through a @RestController endpoint. Never expose the API key in frontend code — the key must stay on the server.

11

Why is it important to keep your OpenAI API key secure when integrating with Spring Boot?

Consider what could happen if someone else gets access to your key.

MODEL ANSWER

Your OpenAI API key is billed to your account — anyone who obtains it can make unlimited calls at your expense, potentially generating thousands of dollars in charges. If exposed in a public GitHub repository, bots scan for keys within seconds and abuse them. Keep the key in environment variables or a secrets manager, never in source code or application.properties committed to version control. In Spring Boot, read it with '@Value("\${openai.api.key}")'. Add application.properties (containing secrets) to .gitignore. Rotate the key immediately if you suspect exposure.

12

In a Spring Boot project, how would you create a REST endpoint that accepts a user question and returns a response from the OpenAI API?

Think about controllers, request mapping, and service classes.

MODEL ANSWER

Create a DTO: `public class QuestionRequest { String question; }`. In a `@RestController`, map `'@PostMapping("/ask")'` to a method that takes `'@RequestBody QuestionRequest req'` and calls `'openAIService.ask(req.getQuestion())'`. The `OpenAIService` builds the JSON payload with the user's question as a message, calls the OpenAI API via `HttpClient`, parses the response, and returns the answer string. The controller wraps it in `ResponseEntity.ok(answer)`. This keeps the API key and HTTP logic inside the service, not the controller.

13

Write a Spring Boot controller method that receives a POST request with a question and returns the OpenAI API's answer as the response.

MODEL ANSWER

```
@RestController @RequestMapping("/api") public class ChatController { @Autowired private OpenAIService openAIService; @PostMapping("/ask") public ResponseEntity<String> ask(@RequestBody Map<String,String> body) { String question = body.get("question"); String answer = openAIService.getAnswer(question); return ResponseEntity.ok(answer); } }
```

The `@RequestBody` binds the incoming JSON. `OpenAIService.getAnswer()` makes the HTTP call to OpenAI and returns the model's reply as a String. Return 200 OK with the answer in the body.

14

How would you test if your 'Ask CampusBot' REST endpoint is correctly calling the OpenAI API and returning the expected output?

MODEL ANSWER

In Postman, send POST to `/api/ask` with body `{ "question": "What are the admission requirements?" }`. Confirm a 200 response with a coherent answer. For unit tests, mock the `OpenAIService` using Mockito: `'when(openAIService.getAnswer("question")).thenReturn("mocked answer")'` and verify the controller returns it. For integration tests, use Wiremock to simulate the OpenAI endpoint locally, avoiding real API calls in CI. Check for error handling by passing an empty question and verifying a 400 response, or by simulating a 429 from Wiremock and confirming a fallback response.

15

What is LangChain4J, and how does it help when building AI chatbots in Java?

Think about how it interacts with language models.

MODEL ANSWER

LangChain4J is a Java library that simplifies building LLM-powered applications by abstracting the HTTP calls, prompt management, memory, and chaining logic. Instead of manually building JSON payloads and parsing responses, you declare an AI service interface and annotate methods with `@UserMessage`, and LangChain4J calls the model and returns the response as a typed Java object. It also provides built-in support for conversation memory, prompt templates, RAG (retrieval-augmented generation), and tool use — features that would take hundreds of lines to implement manually.

16

Explain what a prompt template is in the context of LangChain4J and why it is useful.

MODEL ANSWER

A prompt template is a reusable string with placeholders that are filled at runtime with actual values. For example: 'You are a helpful campus assistant. Answer the following question about {{college}}: {{question}}'. LangChain4J replaces '{{college}}' and '{{question}}' with the actual values before sending the prompt to the model. Templates are useful because they let you control the model's context and behaviour consistently across all calls, personalise responses without rebuilding the prompt string every time, and version-control the prompt text separately from the code logic.

17

Describe the process of creating a simple FAQ chatbot using LangChain4J and OpenAI API in a Java application.

MODEL ANSWER

Add the LangChain4J and OpenAI dependency to pom.xml. Configure the OpenAI model: 'OpenAiChatModel model = OpenAiChatModel.builder().apiKey(apiKey).modelName("gpt-4o-mini").build()'. Define an AI service interface with @AiService and annotate a method '@UserMessage("Answer this campus FAQ: {{question}}") String answer(String question)'. LangChain4J creates the implementation: 'CampusBot bot = AiServices.create(CampusBot.class, model)'. Call 'bot.answer("What are the library hours?")' and the model responds. Expose this through a Spring Boot REST controller.

18

If you want your CampusBot to answer college-specific FAQs, how would you structure the prompt templates in LangChain4J?

MODEL ANSWER

Use a system-level prompt to set the bot's identity and scope: 'You are CampusBot, the official AI assistant for TOPS Technologies. Only answer questions related to courses, fees, placement, and campus events. If the question is out of scope, politely say so.' Use @SystemMessage for this fixed context. Add @UserMessage for the dynamic question: '{{question}}'. Optionally prepend the most relevant FAQ entries from a database (RAG pattern) so the model has factual context. This combination keeps answers accurate, on-topic, and branded.

19

How does LangChain4J simplify chaining multiple prompts or API calls for a chatbot?

Consider how it manages sequences of operations.

MODEL ANSWER

LangChain4J provides a ConversationMemory abstraction that automatically maintains conversation history — each call to the AI service includes previous messages so the model has context. For chaining, you can compose multiple AI service calls in sequence in your Java code, passing the output of one as input to the next. Tools (Java methods annotated @Tool) let the model call your code when it needs real data (e.g., fetching current timetable from the database) and incorporate the result into its response — all managed by LangChain4J without manual JSON handling.

20

Suppose your chatbot is returning irrelevant answers from the OpenAI API. What steps would you take to debug and improve the responses?

Think about prompt engineering, logging, and API parameters.

MODEL ANSWER

First, log the exact prompt being sent to the API — often the system message is too vague. Sharpen the system prompt to restrict the model to specific topics and instruct it to say 'I don't know' for out-of-scope questions. Lower the temperature parameter (e.g., 0.2) to make responses more focused and deterministic. Check if conversation history is too long, causing the model to lose track of the current question. Add few-shot examples of good question-answer pairs to the prompt. If a specific question consistently fails, add it as a hardcoded rule before calling the model.

21

Compare the process of calling the OpenAI API directly via HttpClient and using LangChain4J as a wrapper. What are the advantages of each approach?

MODEL ANSWER

Direct HttpClient gives you full control — you see exactly what JSON is sent and received, there are no hidden abstractions, and it works with any HTTP-based AI API without a specific library. It is good for learning and simple one-shot calls. LangChain4J reduces boilerplate dramatically — no JSON building, no response parsing, automatic memory management, tool integration, and easy swapping between model providers. It is better for production chatbots with conversation history, complex chaining, or RAG. Start with direct HttpClient to understand the API, then migrate to LangChain4J for maintainability.

22

How would you use an AI-powered code assistant like GitHub Copilot or ChatGPT to help you write or debug your Java code that integrates with a REST API? What would you do to ensure the code is correct before using it in your project?

Think about reviewing, testing, and understanding the AI-generated code.

MODEL ANSWER

Copilot can suggest the HttpClient request setup, Jackson parsing code, and error handling as you type. ChatGPT can explain a 401 error from OpenAI, suggest the correct Authorization header format, or generate a complete service class. Before using the generated code: read every line to understand what it does, check that API keys are read from environment variables (not hardcoded), verify the JSON field names match the actual API documentation, write a unit test with a mocked HTTP response to confirm parsing logic, and run the code against the real API in a sandbox environment.

23

Imagine you want to quickly generate prompt templates or sample chatbot responses for your Java project. How could an AI tool help, and what would you check before using its suggestions?

MODEL ANSWER

ChatGPT can draft system prompts, user message templates, and even simulate model responses to test how a chatbot would behave. You can iterate on prompt wording quickly by asking ChatGPT to improve a draft. Before using the suggestions: test the final prompt against the actual OpenAI API to confirm the real model responds as expected (simulated responses from ChatGPT are not the same as production model behaviour). Check that the prompt does not inadvertently allow the model to go off-topic. Review with domain experts (e.g., admissions staff for a campus bot) to verify factual accuracy.

24

Integrate your knowledge: Describe how you would build a Spring Boot endpoint that takes a user question, sends it to OpenAI via LangChain4J, and returns the answer to the user.

MODEL ANSWER

Configure LangChain4J's OpenAiChatModel with the API key from application.properties and a @Bean method. Define an AiService interface CampusBot with a '@SystemMessage' persona and '@UserMessage("{{question}}") String answer(String question)' method. Create a @Service that holds the CampusBot instance via AiServices.create(). Expose it through '@RestController @PostMapping("/chat")' that reads the question from the request body, calls campusBotService.answer(question), and returns the response in a ResponseEntity. Add @CrossOrigin for the React frontend and store the API key as an environment variable.

25

Suppose you want to log every question and answer exchanged between users and your CampusBot chatbot. Which parts of your Java application would you modify, and how would you implement this logging?

Think about controller, service, and possible use of a database or log files.

MODEL ANSWER

In the CampusBotService, after calling campusBot.answer(question), save a ChatLog entity (userId, question, answer, timestamp) via ChatLogRepository.save(). Alternatively, add a @Around AOP advice on the answer() method to intercept all calls and log automatically without touching the service. For lightweight logging without a database, use SLF4J: 'log.info("Q: {} A: {}", question, answer)' and configure log appenders to write to a file. The key is to log in the service layer — not the controller — so all callers (REST, scheduled jobs, etc.) are covered.